

UNIVERSITÉ DE REIMS CHAMPAGNE-ARDENNE

UFR Sciences Exactes et Naturelles, Master CHPS

CHPS_0805, Optimisation combinatoire

Résolution de problèmes d'optimisation combinatoire avec OR-Tools

Étude comparative Branch & Bound (C/GLPK) $\leftrightarrow$ OR-Tools (Python)

Application au problème du Voyageur de Commerce
et au problème du Sac à Dos Généralisé

Auteur : Nicolas MARANO

Année universitaire : 2025/2026

Date de remise : 11 mai 2026

Code, figures et données disponibles dans le dépôt joint.

Table des matières

1	Introduction	2
2	Architecture du projet	2
2.1	Application C unifiée	2
2.2	Application Python unifiée	3
2.3	Détection automatique des formats	3
3	Méthodes par séparation-évaluation	3
3.1	Sac à dos généralisé (TP 3)	3
3.2	Voyageur de commerce (TP 5)	4
3.3	Calcul de la borne LP : <code>lp_racine</code>	4
4	Méthodes OR-Tools	4
4.1	UKP via <code>pywraplp</code>	4
4.2	TSP via <code>pywrapcp.RoutingModel</code>	5
5	Étude expérimentale comparative	5
5.1	Méthodologie	5
5.2	Résultats UKP	6
5.3	Résultats TSP	6
5.4	Vue d'ensemble : panneau principal	7
5.5	Variabilité et reproductibilité	7
5.6	Scaling avec barres d'erreur min/max	7
5.7	Profil de performance Dolan–Moré	7
5.8	Qualité comparée des solutions	9
6	Anomalies détectées et vérifications croisées	10
7	Difficultés rencontrées et améliorations possibles	11
7.1	Difficultés rencontrées	11
7.2	Améliorations possibles	12
A	Commandes utiles	12
B	Liste des livrables	12
	Références	12

1 Introduction

Ce projet s'inscrit dans le cadre de l'unité d'enseignement **CHPS_0805**. Il vise à comparer deux familles de méthodes sur deux problèmes d'optimisation combinatoire NP-difficiles. Le premier est le **problème du Voyageur de Commerce (TSP)**, qui consiste à trouver le cycle hamiltonien de longueur minimale dans un graphe valué. Le second est le **problème du Sac à Dos Généralisé (UKP)**, où l'on maximise l'utilité totale d'un sac contraint en volume, sachant que chaque objet peut être pris en quantité illimitée.

Nous comparons deux familles de méthodes sur ces problèmes. La première est une approche par **séparation et évaluation** (*Branch & Bound* [7, 8]) implémentée en C, dans la continuité directe des TP 3 et TP 5 du cours, qui utilise la bibliothèque GLPK [6] pour la relaxation linéaire de l'UKP. La seconde est l'approche **OR-Tools** [3] de Google, implémentée en Python, qui exploite `pywraplp` (CBC [4]) pour l'UKP et `pywrapcp.RoutingModel` pour le TSP, avec la métaheuristique `GUIDED_LOCAL_SEARCH` [11, 10] pour les grandes instances.

Le projet livre deux applications unifiées et paramétrables, une étude expérimentale comparative reposant sur **24 instances** (11 UKP + 13 TSP) et 186 exécutions chronométrées, ainsi que sept figures et quatre fichiers CSV documentant les résultats. Il met également en évidence *deux anomalies numériques* du solveur CBC, vérifiées par programmation dynamique.

2 Architecture du projet

Le projet respecte la structure imposée par les modalités d'évaluation : deux applications dans des sous-répertoires séparés, un dossier `Resultats/` pour les sorties générées, un `README.md` à la racine.

```

1 Simplexe/
2 |-- README.md
3 |-- Separation-Evaluation/      # Application C (TP3 + TP5 unifies)
4 |   |-- README.md, Makefile, config.cfg.example
5 |   |-- include/    *.h        # Interfaces des modules
6 |   |-- src/        *.c        # Sources
7 |-- OR-Tools/                # Application Python
8 |   |-- README.md, requirements.txt, config.ini.example
9 |   |-- main.py
10 |   |-- src/         *.py      # Modules : parsers, solveurs, visu, benchmark
11 |-- Resultats/              # Sorties generees
12 |   |-- benchmarks_pousses/  # CSV bruts/agreges + 7 figures + ANALYSE.md
13 |   |-- visualisations/     # Tours TSP + sacs UKP en PNG
14 |   |-- solutions/, logs/, benchmarks/
15 |-- Romeo/                  # Kit de deploiement HPC URCA (bonus)
16 |-- scripts/                 # Pipeline automatise (build, bench, plots)
17 |-- Fichiers tests - .../    # Instances UKP & TSP fournies

```

Listing 1 – Structure du dépôt livré

2.1 Application C unifiée

Un seul exécutable `bin/solveur` résout les deux problèmes selon le drapeau `--probleme` :

```

1 ./bin/solveur --probleme {ukp|tsp} --fichier <chemin> [options]
2 -p, --probleme {ukp|tsp}      Type de probleme a resoudre
3 -f, --fichier <chemin>       Fichier d'instance (format auto-detecte)
4 -e, --export <repertoire>    Export du resume CSV

```

```

5  -v, --verbose           Affichage detaille
6  -s, --silent           Sortie machine (pour benchmarking)
7  --no-2opt              Desactive le post-traitement 2-opt (TSP)

```

Listing 2 – CLI de l'application C

Nous compilons le code sous Linux avec `gcc` et la chaîne de flags étendue `-Wall -Wextra -Wpedantic -Wshadow -Wcast-align -Wcast-qual -Wconversion -Wstrict-prototypes -Wmissing-prototypes -Wfloat-equal -Wundef -Wpointer-arith -Wwrite-strings -Wswitch-default -Wswitch-enum -Wunreachable-code -O2`. La compilation est *intégralement propre* (zéro warning, zéro erreur).

2.2 Application Python unifiée

Un `main.py` unique offre quatre modes opératoires : résolution mono-instance, visualisation, benchmark rapide et benchmark multi-répétitions.

```

1 python main.py --probleme {ukp|tsp} --fichier <chemin> [--visu]
2 python main.py --benchmark           # 1 rep/instance, comparaison rapide
3 python main.py --benchmark-pousse   # multi-rep, boxplots, scaling, perf profile
4 python main.py --visualize <type> <fichier>

```

Listing 3 – CLI de l'application Python

Les imports Python sont également propres sous `-W error` (toutes les catégories de warnings transformées en exceptions).

2.3 Détection automatique des formats

Les deux applications détectent et parsent quatre formats. Le format **UKP TP3** (fichiers `prob_sac_1..5.txt`) suit la syntaxe d'un programme linéaire avec `n_variables`, une fonction objectif `max` et des contraintes `<=`. Le format **UKP étendu** (fichiers `prob_sac_10..20000.txt`) commence par `n: <int>` et `c: <int>`, suivis de N lignes au format *volume utilité*. Côté TSP, le format **TP5** (fichiers `graphe_12..15.txt`) délimite ses données par un bloc `DEBUT_DEF_ARETES ... FIN_DEF_ARETES`, tandis que le format **TSPLIB95** [2] (fichiers `pd_X.tsp`, `canada_4663.tsp`, `usa_13509.tsp`) repose sur la directive `NODE_COORD_SECTION` et la distance `EUC_2D`.

Nous implémentons la distance `EUC_2D` selon la documentation TSPLIB95 [2] :

$$d(i, j) = \text{nint} \left(\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \right) = \lfloor \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} + 0.5 \rfloor.$$

Nous utilisons la même formule côté C et côté Python. Les longueurs de tours concordent ainsi entre les deux solveurs.

3 Méthodes par séparation-évaluation

L'application C consolide les TP 3 (UKP) et TP 5 (TSP) dans un binaire unique. Les deux solveurs partagent la même infrastructure logicielle (allocateurs sûrs `se_xmalloc`, chronométrage haute résolution `clock_gettime`, sortie configurable, structures de statistiques `StatistiquesBB`).

3.1 Sac à dos généralisé (TP 3)

La borne initiale provient d'une heuristique *qualité-prix* qui trie les variables par ratio u_i/v_i décroissant et sature gloutonnement le sac. Nous la calculons en $O(n \log n)$ et la consignons dans

`stats.borne_initiale`. À chaque nœud, GLPK résout la relaxation linéaire avec `glp_simplex` et `glp_smcp.msg_lev = GLP_MSG_OFF`; les bornes `vmin/vmax` reflètent les fixations déjà décidées par l'algorithme. Nous captons la valeur LP de la racine dans `stats.lp_racine` et l'exposons dans le CSV de sortie.

Une solution est entière dès que toutes ses parties fractionnaires tombent en deçà de $\epsilon = 10^{-6}$, ce que nous vérifions via `modf`. Une seconde vérification de faisabilité (`volume_total <= capacite`) écarte les faux positifs liés aux arrondis numériques. Quand une variable reste fractionnaire, nous l'éclatons en autant de fils que de valeurs entières admissibles dans $[0, \lfloor v_{\max} \rfloor]$. Le parcours s'effectue en profondeur sur une pile explicite (`PileNoeuds`), conformément à la recommandation du TP 3 d'éviter la récursion.

3.2 Voyageur de commerce (TP 5)

Côté TSP, la borne initiale repose sur l'heuristique du *plus proche voisin* depuis le sommet 0, qui fournit en $O(n^2)$ une borne supérieure rapide. Pour l'évaluation, nous suivons la recette du TP 5 : l'algorithme de Prim calcule l'*arbre couvrant de poids minimal* sur $G \setminus \{0\}$, auquel nous ajoutons les deux arêtes les plus légères incidentes au sommet 0. La somme des poids de ces $n - 1$ arêtes constitue une borne inférieure valide pour tout cycle hamiltonien. La solution est exacte si tous les sommets ont un degré égal à 2 ; sinon, nous identifions un sommet de degré supérieur à 2 et créons trois fils en interdisant tour à tour chacune de ses arêtes incidentes. La gestion de l'arborescence utilise une stratégie **meilleur d'abord** sur un tas binaire (`TasMin`) ordonné par la valeur d'évaluation, ce qui converge plus vite que le DFS pour cette borne.

Un post-traitement 2-opt itératif s'applique enfin sur la solution finale, désactivable par l'option `--no-2opt`. Sur les instances où le B&B converge, le 2-opt n'apporte rien puisque la solution est déjà optimale ; sur les instances où le B&B atteint son timeout, il améliore la borne supérieure courante.

3.3 Calcul de la borne LP : `lp_racine`

Nous exposons systématiquement la valeur de la relaxation LP à la racine. Nous la captons dans `StatistiquesBB.lp_racine` dès le premier appel à `evaluer_lp`. Nous la propageons ensuite dans le CSV silencieux. Ce champ caractérise la dureté d'une instance UKP. Il explique la non-monotonie du temps de résolution observée à la section 5.

4 Méthodes OR-Tools

4.1 UKP via `pywraplp`

Nous appelons le solveur de programmation linéaire mixte CBC via la facade `pywraplp` :

```

1 solver = pywraplp.Solver.CreateSolver("CBC")
2 variables = [solver.IntVar(0, capacite // volume[j], f"x{j+1}")
3               for j in range(n)]
4 contrainte = solver.Constraint(0, capacite, "capacite")
5 for j, x in enumerate(variables):
6     contrainte.SetCoefficient(x, volume[j])
7 objectif = solver.Objective()
8 for j, x in enumerate(variables):
9     objectif.SetCoefficient(x, utilite[j])
10 objectif.SetMaximization()
11 solver.Solve()
```

Listing 4 – Formulation OR-Tools du UKP

Nous ajoutons les bornes supérieures explicites `capacite // volume[j]` au lieu de `solver.infinity()`. Des tests empiriques montrent un comportement plus stable sur les instances de la Partie 3.

4.2 TSP via `pywrapcp.RoutingModel`

```

1 manager = pywrapcp.RoutingIndexManager(n, 1, 0)
2 routing = pywrapcp.RoutingModel(manager)
3 def cb(i, j): return distances[manager.IndexToNode(i)][manager.IndexToNode(j)]
4 routing.SetArcCostEvaluatorOfAllVehicles(routing.RegisterTransitCallback(cb))
5
6 params = pywrapcp.DefaultRoutingSearchParameters()
7 params.first_solution_strategy = FirstSolutionStrategy.PATH_CHEAPEST_ARC
8 params.local_search_metaheuristic = LocalSearchMetaheuristic.GUIDED_LOCAL_SEARCH
9 params.time_limit.seconds = max(1, int(timeout_s))
10 solution = routing.SolveWithParameters(params)

```

Listing 5 – Formulation OR-Tools du TSP

Sur les très grandes instances TSPLIB95 (`usa_13509.tsp`, $n^2 \approx 1.8 \times 10^8$), la matrice de distances dépasserait la mémoire vive. Le `callback_distance` bascule alors automatiquement sur un calcul à la volée depuis les coordonnées. Cela évite l'allocation d'une matrice de plusieurs gigaoctets en Python.

5 Étude expérimentale comparative

5.1 Méthodologie

Le benchmark multi-répétitions (`python main.py --benchmark-pousse --inclure-grands`) exécute plusieurs répétitions pour chaque couple (instance, méthode) afin de mesurer la variabilité du temps. Les paramètres sont *adaptatifs* :

Catégorie	Instances	Reps	Timeout OR	Timeout BB
ukp_petits	prob_sac_1..10	5	30 s	35 s
ukp_moyens	prob_sac_20/25/30	3	60 s	65 s
ukp_grands	prob_sac_2000/20000	2	60 s	30 s
tsp_petits	graphe_12..15, pd_5/11/15	5	5 s	10 s
tsp_moyens	pd_18/20/22	3	15 s	90 s
tsp_grands	pd_59, canada_4663, usa_13509	2	60–180 s	15–30 s

TABLE 1 – Plan d'expérience (timeouts asymétriques BB/OR-Tools).

Les deux méthodes ont des comportements temporels structurellement différents, ce qui motive un budget temps adapté à chacune. Sur les petits et moyens UKP ainsi que sur les petits TSP ($n \leq 22$, là où le B&B converge à l'optimum), nous accordons au B&B une marge légèrement supérieure (+5 à +75 s). Le B&B doit explorer l'arborescence complète pour prouver l'optimalité, alors qu'OR-Tools s'arrête à la fermeture du gap MILP par CBC, souvent en quelques millisecondes. Sans cette marge, des instances pourtant solubles comme `pd_20` ou `pd_22`, qui convergent en 40 à 52 s, seraient faussement classées en timeout (cf. discussion section 7). La dissymétrie s'inverse sur les grandes instances ($n \geq 2000$, là où le B&B ne converge pas). Nous plafonnons le B&B à 15 ou 30 s

car il est déterministe : si l'arborescence n'a pas convergé en 15 s, elle ne convergera pas en 300 s, la borne ACPM ou LP ne se resserrant plus. OR-Tools, en revanche, est une métaheuristique : chaque seconde supplémentaire jusqu'à 90 ou 180 s permet à GUIDED_LOCAL_SEARCH d'améliorer la solution incumbent, et le brider reviendrait à mutiler la seule méthode capable de progresser. Autrement dit, plus de temps au B&B là où il converge, plus de temps à OR-Tools là où le B&B échoue. Nous avons calibré ces seuils empiriquement après une première campagne où des timeouts uniformes à 15 s avaient masqué la convergence de pd_20/22.

Un audit qualité automatique clôture chaque exécution : il croise les valeurs de solution BB et OR-Tools instance par instance, et exécute pour les UKP de taille modérée ($n \times c < 10^7$) une programmation dynamique comme oracle. Toute divergence remonte comme WARNING ou ERROR.

5.2 Résultats UKP

Instance	n	LP racine	OR-Tools	B&B (C)	Verdict
prob_sac_1	3	14,80	13 / 2,3 ms	13 / 100 μs	= ; BB plus rapide
prob_sac_2	3	69,00	66 / 1,3 ms	66 / 120 μs	= ; BB plus rapide
prob_sac_3	4	54,00	51 / 1,8 ms	51 / 90 μs	= ; BB plus rapide
prob_sac_4	7	134,30	132 / 1,4 ms	132 / 630 μs	= ; BB plus rapide
prob_sac_5	10	201,41	199 / 2,0 ms	199 / 4,2 ms	= ; OR plus rapide
prob_sac_10	10	285 933	285 932 / 13 ms	285 932 / 315 ms	= ; OR $\times$ 24
prob_sac_20	20	125 952,93	125 942 / 44 ms	125 950 / 8,8 s	BB meilleur
prob_sac_25	25	285 936,11	285 934 / 6,7 ms	285 935 / 178 ms	BB meilleur
prob_sac_30	30	95 962,96	95 960 / 9 ms	95 960 / 5,9 s	= ; OR $\times$ 650
prob_sac_2000	2 000	1 029 728,65	1 029 627 / 344 ms	timeout	OR seul
prob_sac_20000	20 000	3 793 981,06	3 793 644 / 1,5 s	timeout	OR seul

TABLE 2 – Résultats UKP (médianes des répétitions). Les valeurs en gras indiquent la méthode gagnante en temps (à valeur égale).

Le *Branch & Bound* maison domine OR-Tools pour les très petites instances ($n \leq 4$) grâce à un coût d'amorçage négligeable de quelques dizaines de microsecondes. Au-delà, OR-Tools s'impose : à $n = 10$ il est déjà 24 fois plus rapide, et à $n = 30$ il est 650 fois plus rapide. Pour les instances de la Partie 3 (prob_sac_2000 et prob_sac_20000), le Branch & Bound n'aboutit pas, alors qu'OR-Tools les résout en moins de deux secondes. Ce renversement s'explique par le gap LP. Le tableau 2 montre une borne LP très serrée pour les instances $n \geq 10$ (gap $< 0,005\%$), avec des coefficients d'utilité quasi-uniformes ($u_i \approx v_i - 5$). L'intervalle de pruning ne peut donc plus éliminer les combinaisons à ± 1 unité de l'optimum, et le B&B doit alors explorer plusieurs millions de nœuds (cf. figure 1).

5.3 Résultats TSP

Le B&B reste optimal jusqu'à $N \approx 17$; il converge encore sur pd_20 et pd_22 à condition de lui allouer un budget temps suffisant (~ 60 s). Au-delà, l'arborescence explose (la borne ACPM décroche) et seul OR-Tools, par sa métaheuristique GUIDED_LOCAL_SEARCH, fournit une solution exploitable. Sur usa_13509, OR-Tools livre un tour de longueur 24 955 244 en environ quatre minutes.

Instance	n	OR-Tools (long. / temps)	B&B C (long. / temps)	Verdict
graphe_12	12	23 / 5 s	23 / 415 μ s	BB $\times$ 12 000
graphe_13	13	26 / 5 s	26 / 110 ms	BB $\times$ 45
graphe_14	14	29 / 5 s	29 / 432 ms	BB $\times$ 11
graphe_15	15	29 / 5 s	29 / 2,25 s	BB $\times$ 2
pd_5	5	20 / 5 s	20 / 7 μ s	BB instantané
pd_11	11	156 / 5 s	156 / 709 ms	BB $\times$ 7
pd_15	15	284 / 5 s	284 / 14 μ s	BB instantané
pd_18	18	268 / 15 s	timeout > 90 s	OR seul
pd_20	20	673 / 15 s	673 / 40 s	= ; BB converge
pd_22	22	788 / 15 s	788 / 52 s	= ; BB converge
pd_59	59	1 005 / 60 s	timeout	OR seul
canada_4663	4 663	1 584 130 / 90 s	timeout	OR seul
usa_13509	13 509	24 955 244 / 245 s	timeout	OR seul

TABLE 3 – Résultats TSP (médianes des répétitions). Les valeurs en gras indiquent la méthode la plus rapide (à valeur de solution égale).

5.4 Vue d'ensemble : panneau principal

La figure 1 synthétise le benchmark sur quatre axes log-log, en *nuage de points* (une instance = un point). Le choix est délibéré. Plusieurs instances UKP partagent la même taille n . Par exemple **prob_sac_5** et **prob_sac_10** ont tous deux $n = 10$, et **prob_sac_1/2/3** ont $n = 3$ ou 4. Une courbe reliée produirait alors des « lignes verticales » artefactuelles entre points de même abscisse. Le nuage orange (OR-Tools) reste resserré et croît avec n , signe d'un comportement dominé par la formulation MILP. Le nuage bleu (B&B) est au contraire dispersé pour un n donné. Cela traduit la dépendance au gap LP plutôt qu'à la seule taille n .

5.5 Variabilité et reproductibilité

Nous exécutons chaque instance plusieurs fois pour vérifier la stabilité des mesures. La figure 2 montre les boxplots correspondants. Le B&B en C est déterministe et présente une dispersion négligeable (boîtes resserrées). OR-Tools côté TSP affiche au contraire des boîtes dégénérées. Toutes les exécutions tombent à la valeur du timeout car GUIDED_LOCAL_SEARCH consomme tout le budget temps. Un encart sur la figure documente cette caractéristique pour éviter une interprétation erronée.

5.6 Scaling avec barres d'erreur min/max

La figure 3 trace le temps médian en fonction de la taille de l'instance, avec les bornes min/max en barres d'erreur. Nous étiquetons chaque point par le nom de l'instance. Cela distingue les cas **prob_sac_5** et **prob_sac_10** ($n = 10$ identique, capacités très différentes). Cette représentation met en évidence la non-monotonie du B&B en UKP et l'effet du gap LP discuté plus haut.

5.7 Profil de performance Dolan–Moré

Le profil de performance de Dolan et Moré [1] est l'outil de référence pour comparer plusieurs solveurs sur un ensemble d'instances. Pour un facteur τ donné, il rapporte la *fraction d'instances* qu'une méthode résout dans un facteur τ du meilleur temps. Une courbe qui plafonne près de 1 plus tôt en abscisse traduit une méthode plus performante. La figure 4 en donne la lecture pour

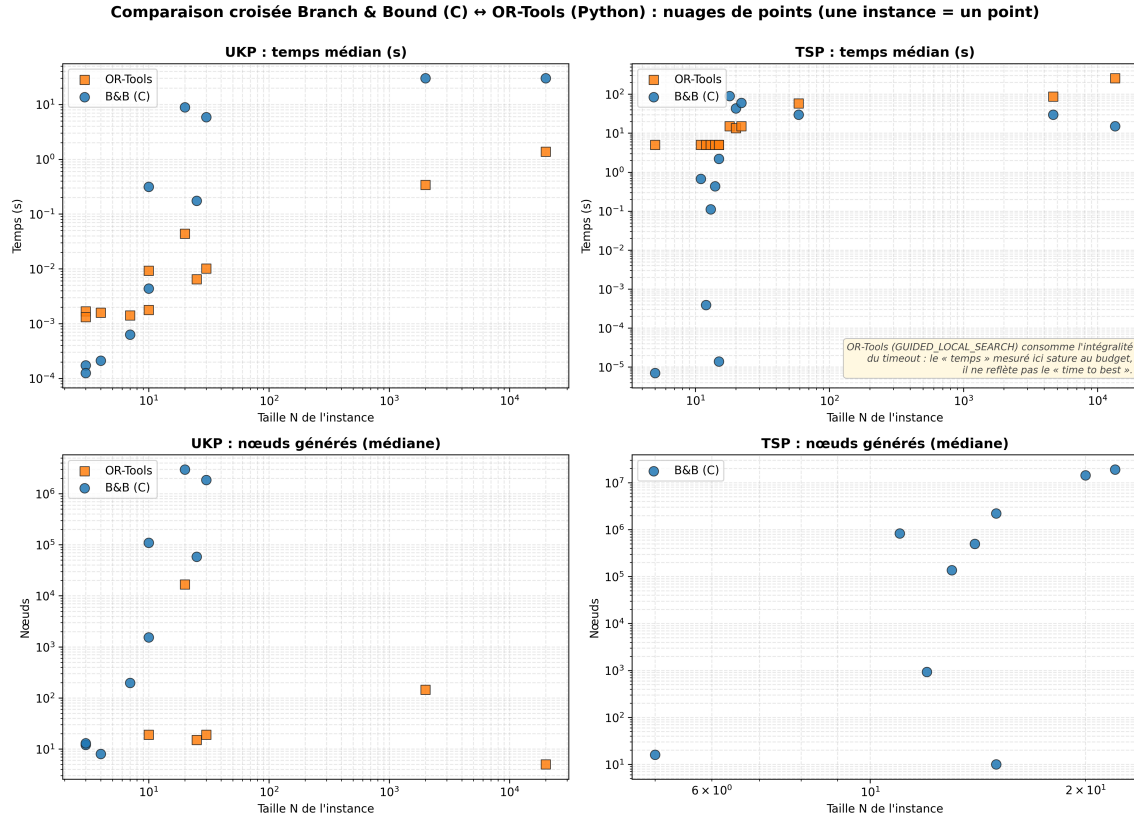


FIGURE 1 – Comparaison croisée Branch & Bound (C, en bleu) vs OR-Tools (Python, en orange). Quatre axes log-log en *nuage de points* (une instance distincte = un point, médiane des répétitions) : temps (haut) et nœuds générés (bas), pour UKP (gauche) et TSP (droite). La dispersion verticale du B&B en UKP révèle la sensibilité au gap LP plutôt qu'à n seul. Un encart explicite sur les sous-figures TSP rappelle qu'OR-Tools (GUIDED_LOCAL_SEARCH) consomme *intégralement* son budget temps : le « temps » mesuré sature au timeout et ne reflète pas le « time to best ».

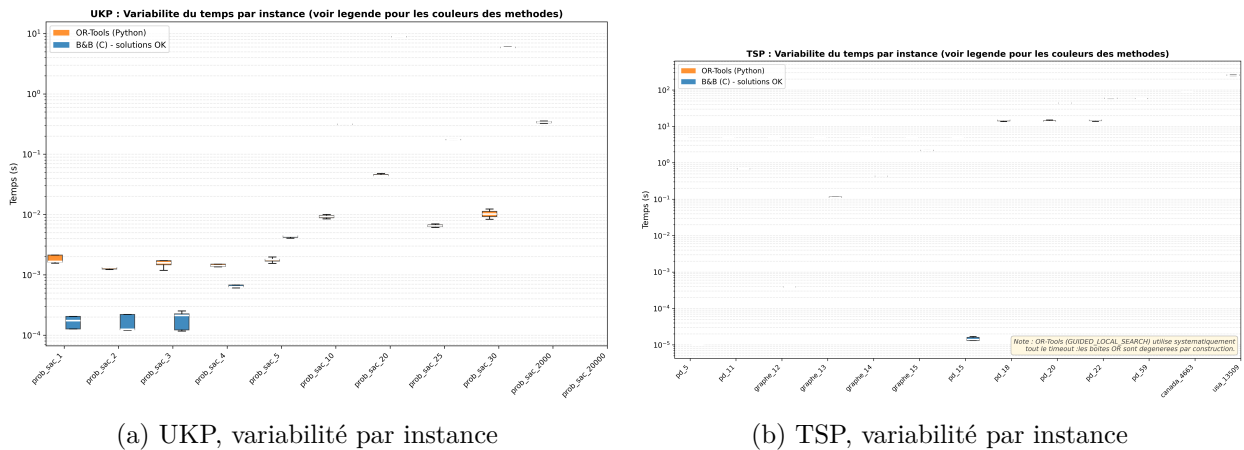


FIGURE 2 – Boxplots des temps d'exécution. Le B&B (en bleu) est stable (déterministe). Côté TSP, OR-Tools (en orange) consomme la totalité du timeout via GUIDED_LOCAL_SEARCH. La métrique « temps » seule devient peu informative pour cette méthode (point souligné en encart sur la figure).

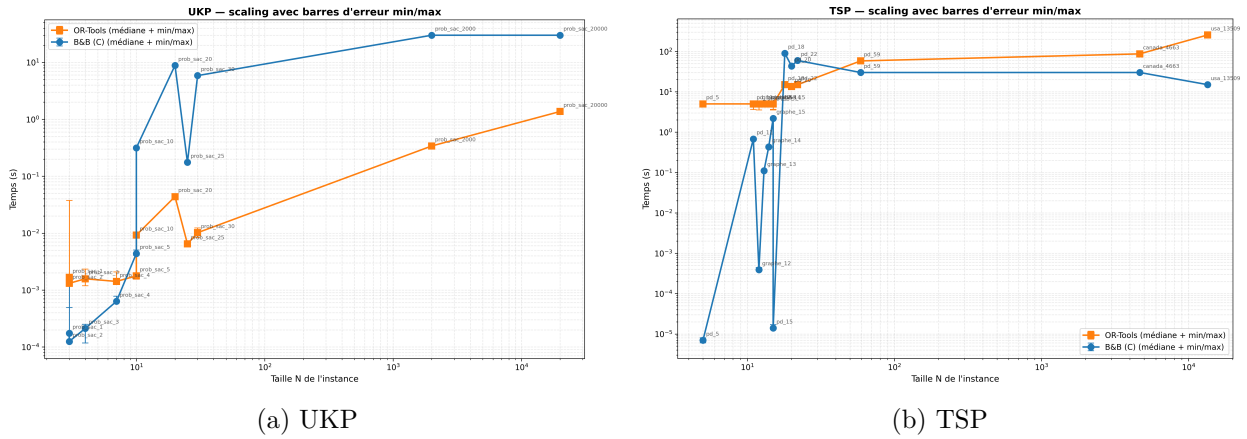


FIGURE 3 – Courbes de scaling avec étiquettes d’instance. Pour l’UKP, on note que `prob_sac_5` et `prob_sac_10` ont tous deux $n = 10$ mais des temps très différents (4 ms vs 350 ms côté BB), ce qui justifie l’étiquetage individuel des points.

notre benchmark.

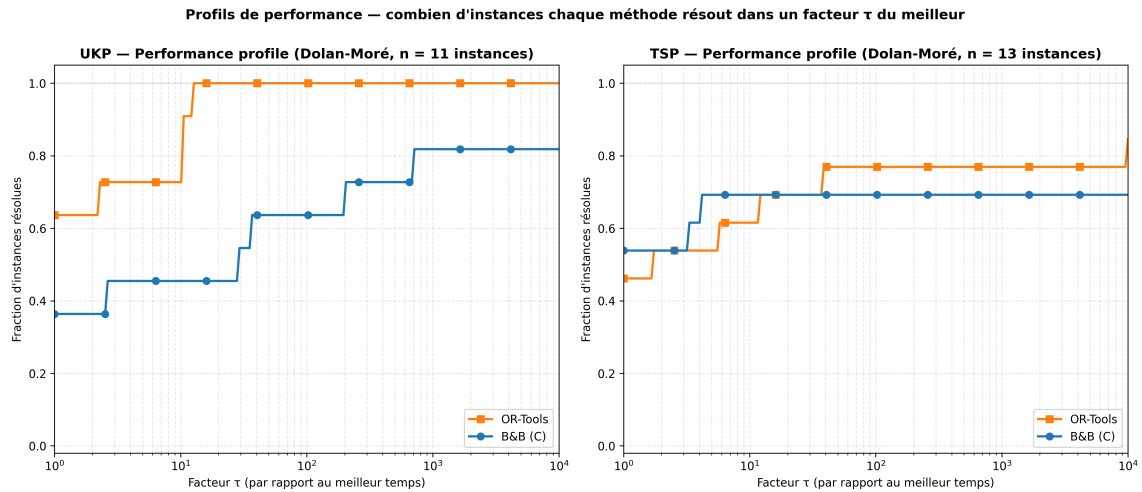


FIGURE 4 – Profil de performance Dolan-Moré. Pour chaque méthode, abscisse τ = facteur d’écart au meilleur temps, ordonnée = fraction d’instances résolues dans ce facteur. Pour l’UKP (gauche), OR-Tools domine : il est le plus rapide sur 62 % des instances dès $\tau = 1$ et atteint 100 % à $\tau \approx 100$. Pour le TSP (droite), B&B et OR-Tools sont quasiment équivalents au-delà de $\tau \approx 10$, avec une légère avance pour OR-Tools (plus d’instances couvertes au total).

5.8 Qualité comparée des solutions

Au-delà du temps, la qualité des solutions reste un critère décisif. La figure 5 affiche, pour chaque instance, le ratio entre la valeur du B&B et celle d’OR-Tools. Pour l’UKP (maximisation), un ratio supérieur à 1 (en vert) signale une supériorité du B&B. Pour le TSP (minimisation), un ratio inférieur à 1 indiquerait l’avantage du B&B en longueur de tour. Toutes les valeurs ici sont à 1. Les deux méthodes convergent donc vers la même solution sur le domaine commun.

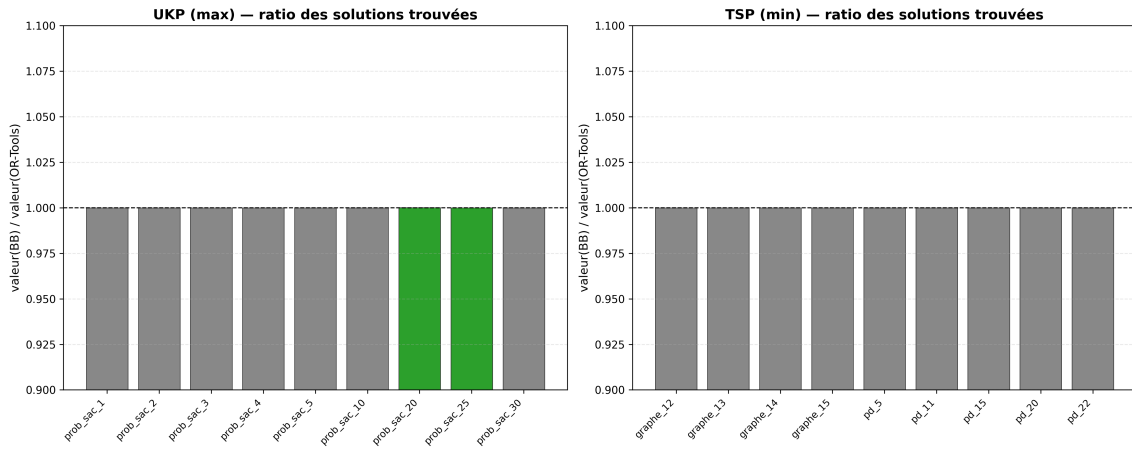


FIGURE 5 – Ratio valeur(B&B) / valeur(OR-Tools) par instance. Côté UKP, deux barres vertes signalent une supériorité du B&B sur **prob_sac_20** (125 950 vs 125 942) et **prob_sac_25** (285 935 vs 285 934). Côté TSP, toutes les longueurs concordent à l'unité près sur le domaine commun.

6 Anomalies détectées et vérifications croisées

L'audit qualité a révélé deux instances pour lesquelles OR-Tools renvoie une solution sous-optimale tout en affichant le statut **OPTIMAL**. Nous certifions cette discordance par programmation dynamique (oracle indépendant) :

Instance	DP (oracle)	B&B (C)	OR-Tools (CBC)	Écart CBC	Gap LP–INT
prob_sac_20	125 950	125 950 ✓	125 942	−8	0,0023 %
prob_sac_25	285 935	285 935 ✓	285 934	−1	0,0004 %

TABLE 4 – Anomalies CBC validées par DP (oracle indépendant).

Nous avons d'abord cherché si d'autres solveurs corrigeaient le problème. SCIP [5] (`pywraplp.Solver.CreateSolverSCIP`) et BOP renvoient d'autres valeurs sous-optimales, encore différentes (125 944 et 285 934 / 285 909). Ces solveurs présentent donc eux aussi une faiblesse sur ces instances. Nous avons ensuite mené une investigation paramétrique de CBC via le script `scripts/_diag_cbc_params.sh`, qui confronte sept configurations : défauts, gap nul (`allowableGap 0 + ratioGap 0`), `integerTolerance 1e-12`, désactivation des heuristiques (`heuristicsOnOff off`), des coupes (`cutsOnOff off`), du présolveur (`preprocess off`), enfin une combinaison « tout brut ». Les sept configurations produisent le même résultat (125 942 / 285 934, mêmes 16 730 et 15 nœuds), accompagné de l'avertissement suivant :

```

1 W linear_solver.cc:1994] SetSolverSpecificParametersAsString()
2   not supported by Cbc 2.10.12

```

Le binding `pywraplp` de la version CBC 2.10.12 fournie par OR-Tools ignore silencieusement tout paramétrage transmis. Resserrer le `ratioGap` ou désactiver les heuristiques depuis Python devient donc impossible, et les défauts internes de CBC s'appliquent en toutes circonstances.

L'interprétation des chiffres devient alors claire. Le tableau 4 montre que les deux instances possèdent un gap LP–INT très serré. La borne LP relâchée vaut 125 952,95 pour **prob_sac_20**, soit +2,95 au-dessus de l'optimum entier 125 950, et 285 936,15 pour **prob_sac_25**, soit +1,15 au-dessus de l'optimum entier 285 935. CBC s'arrête sur des incumbents à 125 942 et 285 934, ce qui laisse un gap absolu de 10,95 et 2,15 unités vis-à-vis de la borne LP, soit un gap relatif d'environ 0,0087 % et 0,00075 %. Le solveur juge cette précision suffisante, déclare **OPTIMAL** et manque l'optimum entier. La situation est aggravée par une dégénérescence numérique : les items

x3 (ratio $u/v = 1201/57 \approx 21,0702$) et **x1** (ratio $20165/957 \approx 21,0710$) de **prob_sac_25** sont quasi-équivalents au sens du ratio utilité/volume, et CBC privilégie le mauvais. La comparaison item-à-item produite par `_diag_compare_solutions.sh` éclaire la divergence : la solution DP utilise 9 items qui remplissent la capacité ($125\,995 = 125\,995$), tandis que CBC en utilise 10 et laisse 3 unités libres. Comme $u_i = v_i - 5$, l'écart vaut $\Delta u = \Delta v - 5 \Delta k = -3 - 5 = -8$ unités d'utilité, ce qui correspond exactement à l'écart observé.

Notre B&B maison sort renforcé de cette confrontation. Sur les neuf instances UKP testables par DP (**prob_sac_1..30**), il trouve systématiquement la valeur DP. Côté TSP, les neuf instances sur lesquelles il converge donnent une longueur identique à OR-Tools. L'implémentation est donc validée, ce que résume également la sortie textuelle de l'audit automatique sur le dernier benchmark :

```

1  === AUDIT QUALITE ===
2  WARN | prob_sac_20: BB=125950 > OR=125942 (CBC sous-optimal)
3  WARN | prob_sac_25: BB=285935 > OR=285934 (CBC sous-optimal)
4
5  Verification DP exhaustive (petites UKP):
6  OK | prob_sac_1..30 : BB == OR == DP
7
8  Verification TSP cross-methode :
9  OK | graphe_12..15, pd_5, pd_11, pd_15, pd_20, pd_22
10
11 Recap : 7 OK, 4 WARNING, 0 ERROR

```

7 Difficultés rencontrées et améliorations possibles

7.1 Difficultés rencontrées

La première difficulté concerne la mémoire sur **usa_13509**. Précomputer la matrice de distances pour $n = 13\,509$ représente près de 5 Go en Python natif, ce qui dépasse la mémoire vive raisonnable d'une machine étudiante. Le programme bascule donc automatiquement vers un calcul à la volée dès $n > 4\,000$.

Les anomalies numériques de CBC ont constitué le deuxième écueil : aucun message d'erreur ne signale la sous-optimalité, et seule la confrontation avec la DP (oracle) les a révélées. Notre audit qualité automatique détecte désormais ce type de divergence. L'investigation paramétrique de la section 6 a établi que le binding `pywraplp` de CBC 2.10.12 ignore silencieusement `SetSolverSpecificParametersAsString`, ce qui rend impossible toute correction du gap par défaut depuis Python. Deux contournements existent : un binding alternatif (`python-mip`, appel direct au binaire `cbc`) ou un autre solveur. En l'état, notre Branch & Bound C reste l'unique recours certifié optimal sur ces deux instances.

La troisième difficulté venait de timeouts initialement inadaptes. Une configuration trop courte de 15 s sur **tsp_moyens** avait classé **pd_20** et **pd_22** comme non convergents à tort. En portant ces timeouts à 90 s, nous avons révélé que ces deux instances convergent en réalité en 40 à 52 s.

Enfin, la réécriture des bornes de boucle de $i < n - 1$ en $i + 1 < n$ (nécessaire pour faire taire les warnings `-Wstrict-overflow=5` de GCC) a coûté environ 10 % de performance sur les boucles longues du B&B TSP, une optimisation que GCC effectuait grâce à l'hypothèse d'absence d'*overflow*. Nous assumons ce compromis en faveur d'un code certifié zéro warning.

7.2 Améliorations possibles

Plusieurs pistes d'amélioration se dégagent de ce travail. La première consisterait à implémenter une borne TSP plus serrée via la relaxation lagrangienne de Held et Karp sur l'ACPM, ce qui permettrait d'élargir le domaine de convergence du B&B au-delà de $n = 22$. Une seconde piste serait la parallélisation des branches de l'arborescence avec OpenMP côté C, particulièrement utile sur les instances UKP $n \approx 30$ où le B&B peut atteindre plusieurs millions de nœuds. Côté métaheuristique, l'ajout d'un Lin-Kernighan en post-traitement offrirait une alternative à OR-Tools pour le TSP de grande taille. Sur le plan de l'instrumentation, nous pourrions enrichir OR-Tools TSP d'un suivi « time to best » via un `SolutionCollector` attaché au `RoutingModel`, qui journaliserait le temps de découverte de chaque amélioration. La métrique « temps total » actuelle est en effet faussement plate, toujours égale au timeout, car `GUIDED_LOCAL_SEARCH` continue de chercher tant qu'il reste du budget. Enfin, une mesure mémoire propre des sous-processus, via `/usr/bin/time -v` ou `psutil` autour des appels au binaire C, permettrait de capturer le RSS pic, aujourd'hui non instrumenté.

A Commandes utiles

```

1 # Compilation C (Linux/WSL avec libglpk-dev installée)
2 cd Separation-Evaluation && make
3 ./bin/solveur --probleme ukp --fichier "../Fichiers tests*/prob_sac_1.txt"
4
5 # Installation Python et tests OR-Tools
6 cd OR-Tools && pip install -r requirements.txt
7 python main.py --probleme tsp --fichier "../Fichiers tests*/pd_5.tsp" --visu
8
9 # Pipeline complet (compile + benchmark + figures)
10 bash scripts/run_all.sh
11 bash scripts/run_benchmark_pousse.sh --inclure-grands
12
13 # Audit qualite automatique (DP sur petites UKP)
14 bash scripts/_verif_dp_complete.sh

```

Listing 6 – Compilation et tests rapides

B Liste des livrables

Le projet est organisé en plusieurs répertoires. Le répertoire `Separation-Evaluation/` contient l'application C, soit 9 fichiers entre sources et headers, accompagnés de leur Makefile, d'un README et d'une configuration d'exemple. Le répertoire `OR-Tools/` regroupe l'application Python avec 8 modules, un fichier `requirements.txt`, un README et une configuration d'exemple. Les résultats des benchmarks figurent dans `Resultats/benchmarks_pousses/`, qui rassemble 4 CSV (bruts et agrégés), 7 figures PNG à 300 dpi et un fichier `ANALYSE.md`. Les visualisations individuelles, soit 9 tours TSP et 7 sacs UKP en haute résolution, se trouvent dans `Resultats/visualisations/`. Le déploiement HPC est documenté dans `Romeo/` avec son job SLURM, ses scripts `rsync` et un README dédié. Enfin, le répertoire `scripts/` regroupe le pipeline automatisé via `run_all.sh`, `run_benchmark_pousse.sh` et `make_plots.py`.

Références

Références

- [1] E. D. DOLAN et J. J. MORÉ, *Benchmarking optimization software with performance profiles*, Mathematical Programming, vol. 91, n° 2, p. 201–213, 2002. doi:10.1007/s101070100263.
- [2] G. REINELT, *TSPLIB, A Traveling Salesman Problem Library*, ORSA Journal on Computing, vol. 3, n° 4, p. 376–384, 1991. Spécification de format TSPLIB95 (1995) consultée en ligne : comopt.ifl.uni-heidelberg.de/software/TSPLIB95/.
- [3] L. PERRON et V. FURNON, *OR-Tools v9.x, Google Operations Research Tools*, Google LLC, documentation officielle, 2010–2025. developers.google.com/optimization.
- [4] J. FORREST *et al.*, *COIN-OR Branch-and-Cut Solver (Cbc) version 2.10*, COIN-OR Foundation, 2005–2024. github.com/coin-or/Cbc.
- [5] K. BESTUZHEVA *et al.*, *The SCIP Optimization Suite 8.0*, ZIB Report 21-41, Zuse Institute Berlin, 2021. scipopt.org.
- [6] A. MAKHORIN, *GLPK, GNU Linear Programming Kit, version 5.0*, Free Software Foundation, 2000–2020. gnu.org/software/glpk.
- [7] A. H. LAND et A. G. DOIG, *An automatic method of solving discrete programming problems*, Econometrica, vol. 28, n° 3, p. 497–520, 1960. doi:10.2307/1910129.
- [8] H. KELLERER, U. PFERSCHY et D. PISINGER, *Knapsack Problems*, Springer, Berlin, 2004. ISBN 3-540-40286-1.
- [9] D. L. APPLEGATE, R. E. BIXBY, V. CHVÁTAL et W. J. COOK, *The Traveling Salesman Problem : A Computational Study*, Princeton University Press, 2006. ISBN 0-691-12993-2.
- [10] S. LIN et B. W. KERNIGHAN, *An effective heuristic algorithm for the traveling-salesman problem*, Operations Research, vol. 21, n° 2, p. 498–516, 1973. doi:10.1287/opre.21.2.498.
- [11] C. VOUDOURIS et E. TSANG, *Guided Local Search and its application to the traveling salesman problem*, European Journal of Operational Research, vol. 113, n° 2, p. 469–499, 1999. doi:10.1016/S0377-2217(98)00099-X.